resume-agent 프로젝트 개선 제안서

☒개요

이 문서는 resume-agent 프로젝트의 현재 아키텍처와 기능을 분석한 후, 실질적인 개선 방향을 제안합니다. 개선안은 **안정성**, **확장성**, **사용자 경험**, **품질 관리** 네 가지 축으로 구성됩니다.

☒개선 로드맵

```
Phase 1: 안정성 강화 (1-2개월)
Phase 2: 사용자 경험 개선 (2-3개월)
Phase 3: 지능형 기능 확장 (3-4개월)
```

☒Phase 1: 안정성 강화

1.1 에러 처리 및 복구 메커니즘

현재 문제점: - 파이프라인 중간 실패 시 전체 재실행 필요 - Codex API 호출 실패 시 대안 부재

개선안:

```
# checkpoint.py - 체크포인트 시스템
class CheckpointManager:
    def save_checkpoint(self, step: str, state: dict):
        """각 단계 완료 시 체크포인트 저장"""
        checkpoint_path = f"{run_dir}/checkpoints/{step}.json"
        with open(checkpoint_path, 'w') as f:
            json.dump(state, f)

    def resume_from_checkpoint(self, step: str):
        """특정 단계부터 재시작"""
        checkpoint_path = f"{run_dir}/checkpoints/{step}.json"
        if os.path.exists(checkpoint_path):
```

```
            return json.load(open(checkpoint_path))
        return None
```

추가 구현: - `resume-agent resume my_run --from coach` 명령어 추가 - 각 단계별 타임아웃 설정 - 자동 재시도 로직 (최대 3회)

1.2 다중 CLI 도구 지원

현재 문제점: - Codex CLI에만 의존하여 도구 선택권 부족 - 사용자 환경에 따라 다른 CLI 도구 선호 가능

개선안:

```
# cli_tool_manager.py - 다중 CLI 도구 지원
class CLIToolManager:
    """사용자가 제공한 CLI 도구 중 선택하여 사용"""

    SUPPORTED_TOOLS = {
        'codex': 'codex',
        'claude': 'claude',
        'gemini': 'gemini',
        'cline': 'cline'
    }

    def __init__(self, tool_name: str = 'codex'):
        self.tool_name = tool_name
        self.tool_command = self.SUPPORTED_TOOLS.get(tool_name)

        if not self.tool_command:
            raise ValueError(f"지원하지 않는 도구: {tool_name}")

    def execute(self, prompt: str) -> str:
        """선택된 CLI 도구로 프롬프트 실행"""
        import subprocess

        cmd = [self.tool_command, '-p', prompt]
        result = subprocess.run(cmd, capture_output=True, text=True)

        if result.returncode != 0:
            raise RuntimeError(f"{self.tool_name} 실행 실패: {result.stderr}")

        return result.stdout
```

사용법:

```
# 기본값(Codex) 사용
resume-agent writer my_run
```

```
# 다른 도구 선택
resume-agent writer my_run --tool claude
resume-agent writer my_run --tool gemini
resume-agent writer my_run --tool cline
```

장점: - 사용자 자유도 향상 (원하는 도구 선택 가능) - API 키 불필요 (CLI 도구 직접 호출) - 단순한 구조 (폴백 로직 없음)

1.3 데이터 검증 강화

개선안:

```
# validators.py - 강화된 검증기
class EnhancedValidator:
    def validate_experience(self, exp: Experience) -> ValidationResult:
        errors = []

        # STAR 구조完整性 검증
        if not exp.situation or len(exp.situation) < 50:
            errors.append("Situation 설명이 부족합니다 (최소 50자)")

        if not exp.task or len(exp.task) < 30:
            errors.append("Task 설명이 부족합니다 (최소 30자)")

        # 구체성 검증 (숫자, 지표 포함 여부)
        if not self._contains_metrics(exp.result):
            warnings.append("결과에 구체적인 수치가 포함되면 더 효과적입니다")

        # 논리적 일관성 검증
        if not self._is_logically_consistent(exp):
            errors.append("STAR 요소 간 논리적 연결이 부족합니다")

        return ValidationResult(errors=errors, warnings=warnings)
```

☒Phase 2: 사용자 경험 개선

2.1 인터랙티브 대화형 인터페이스

현재 문제점: - 일괄 처리 방식으로 중간 개입 어려움 - 사용자 피드백 반영 불편

개선안:

```python
# interactive.py - 대화형 모드
class InteractiveCoach:
    def run(self):
        print("□ 코칭 세션을 시작합니다.")

        while True:
            suggestion = self.generate_suggestion()
            print(f"\n제안: {suggestion}")

            response = input("수정하시겠습니까? (y/n/e=편집): ").lower()

            if response == 'y':
                self.accept_suggestion(suggestion)
            elif response == 'e':
                edited = self.open_editor(suggestion)
                self.accept_suggestion(edited)
            elif response == 'n':
                self.request_alternative()
            elif response == 'q':
                break
```

추가 기능: - `resume-agent coach my_run --interactive` 플래그 추가 - 실시간 미리보기 (터미널 Rich 라이브러리 활용) - undo/redo 기능

2.2 웹 대시보드 (선택사항)

개선안:

```python
# web/dashboard.py - 웹 UI
from fastapi import FastAPI
from fastapi.responses import HTMLResponse

app = FastAPI()

@app.get("/", response_class=HTMLResponse)
async def dashboard():
    return """
    <html>
        <body>
            <h1>resume-agent 대시보드</h1>
            <div id="pipeline-status"></div>
            <div id="preview"></div>
            <div id="editor"></div>
        </body>
    </html>
    """

```

```
@app.get("/api/status/{run_id}")
async def get_status(run_id: str):
    return load_run_state(run_id)

@app.post("/api/approve/{run_id}/{step}")
async def approve_step(run_id: str, step: str):
    return advance_pipeline(run_id, step)
```

기능: - 파이프라인 진행 상태 시각화 - 단계별 결과 미리보기 - 온라인 편집기 - 다운로드 기능

2.3 프로그레스바 및 상태 표시

개선안:

```
# progress.py - 진행 상태 표시
from rich.progress import Progress, SpinnerColumn, TextColumn
from rich.console import Console

class ProgressReporter:
    def __init__(self):
        self.console = Console()

    def run_with_progress(self, steps: list):
        with Progress(
            SpinnerColumn(),
            TextColumn("[progress.description]{task.description}"),
            transient=True,
        ) as progress:
            for step in steps:
                task = progress.add_task(f"실행 중: {step.name}", total=None)
                try:
                    result = step.execute()
                    progress.update(task, description=f"□ {step.name} 완료")
                except Exception as e:
                    progress.update(task, description=f"□ {step.name} 실패: {e}")
                    raise
```

☒Phase 3: 지능형 기능 확장

3.1 벡터 임베딩 기반 검색

현재 문제점: - 키워드 기반 검색으로 의미적 유사도 반영 부족 - 관련성 높은 참고 자료 누락 가능

개선안:

```
# vector_store.py - 벡터 검색 엔진
from sentence_transformers import SentenceTransformer
import chromadb

class VectorKnowledgeBase:
    def __init__(self):
        self.model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
        self.client = chromadb.PersistentClient(path="./kb/vector")
        self.collection = self.client.get_or_create_collection("patterns")

    def index_pattern(self, pattern_id: str, text: str, metadata: dict):
        embedding = self.model.encode(text).tolist()
        self.collection.add(
            ids=[pattern_id],
            embeddings=[embedding],
            metadatas=[metadata]
        )

    def search_similar(self, query: str, n_results: int = 5):
        query_embedding = self.model.encode(query).tolist()
        results = self.collection.query(
            query_embeddings=[query_embedding],
            n_results=n_results
        )
        return results
```

장점: - 의미적 유사도 기반 검색 - 다국어 지원 (한국어/영어) - 오타 및 표현 차이 허용

3.2 자동 품질 평가 시스템

개선안:

```
# quality_evaluator.py - 품질 평가 엔진
class QualityEvaluator:
    def evaluate_draft(self, draft: str, question: str, experience: Experience) ->
QualityScore:
        scores = {}

        # 1. 관련성 평가 (질문-답변 일치도)
        scores['relevance'] = self._calculate_relevance(question, draft)

        # 2. 구체성 평가 (수치, 지표 포함 여부)
        scores['specificity'] = self._calculate_specificity(draft)

        # 3. 논리성 평가 (STAR 구조 충실도)
```

```
        scores['logic'] = self._evaluate_star_structure(draft)

        # 4. 독창성 평가 (표절 및 클리셰 검출)
        scores['originality'] = self._check_originality(draft)

        # 5. 가독성 평가 (문장 길이, 어휘 난이도)
        scores['readability'] = self._calculate_readability(draft)

        # 종합 점수
        overall = sum(scores.values()) / len(scores)

        return QualityScore(overall=overall, details=scores)

    def _check_originality(self, text: str) -> float:
        # 클리셰 패턴 검출
        cliches = [
            "최선을 다하겠습니다",
            "열정적으로 임하겠습니다",
            "팀원들과 소통하며"
        ]

        penalty = sum(1 for cliche in cliches if cliche in text) * 0.1
        return max(0, 1.0 - penalty)
```

3.3 피드백 학습 루프

개선안:

```
# feedback_learner.py - 피드백 학습
class FeedbackLearner:
    def __init__(self):
        self.feedback_db = FeedbackDatabase()

    def record_feedback(self, draft_id: str, feedback: UserFeedback):
        """사용자 피드백 기록"""
        self.feedback_db.save(draft_id, feedback)

        # 피드백 패턴 학습
        if feedback.accepted:
            self._reinforce_pattern(feedback.pattern_used)
        else:
            self._weaken_pattern(feedback.pattern_used)

    def get_recommendation(self, context: dict) -> list:
        """학습된 패턴 기반 추천"""
        similar_contexts = self.feedback_db.find_similar(context)

        # 성공률이 높은 패턴 우선 추천
```

```
        patterns = sorted(
            similar_contexts,
            key=lambda x: x.success_rate,
            reverse=True
        )

        return patterns[:5]
```

☒개선 효과 예측

정량적 지표

지표	현재	Phase 1 후	Phase 2 후	Phase 3 후
파이프라인 성공률	~85%	~95%	~97%	~99%
평균 실행 시간	~10분	~8분	~6분	~4분
사용자 만족도	-	+20%	+40%	+60%
재사용률	-	+15%	+30%	+50%

정성적 효과

Phase 1 (안정성): - 중간 실패 시 재시작 가능 → 사용자 스트레스 감소 - 데이터 검증 강화 → 출력 품질 향상

Phase 2 (UX): - 대화형 인터페이스 → 사용자 통제감 향상 - 실시간 피드백 → 결과 만족도 증가

Phase 3 (지능형): - 벡터 검색 → 더 정확한 참고 자료 매칭 - 품질 평가 → 자동 개선 제안

☒우선순위 매트릭스

개선 항목	긴급도	중요도	난이도	우선순위
체크포인트 시스템	높음	높음	중간	☒☒☒☒☒

개선 항목	긴급도	중요도	난이도	우선순위
다중 CLI 도구 지원	높음	높음	낮음	⛒⛒⛒⛒⛒
인터랙티브 모드	중간	높음	중간	⛒⛒⛒⛒
벡터 검색	낮음	높음	높음	⛒⛒⛒
프로그레스바	중간	중간	낮음	⛒⛒⛒

⛒결론

이 개선 제안서는 resume-agent의 현재 강점을 유지하면서, 실사용자의 피드백과 업계 모범 사례를 반영하여 작성되었습니다.

핵심 개선 방향: 1. **안정성 확보** → 체크포인트, 다중 CLI 도구 지원으로 안정성 향상 2. **사용자 경험 개선** → 인터랙티브 모드, 프로그레스바로 편의성 강화 3. **지능형 기능** → 벡터 검색, 품질 평가로 출력 품질 향상

각 Phase는 독립적으로 실행 가능하며, 우선순위에 따라 조정할 수 있습니다.

제안서 작성: 2026년 3월 20일 기반: resume-agent 종합 분석 보고서